

PA4 CUDA Ray Tracing Profiling & Performance Report

Course: ECE 6360

Assignment: PA4 CUDA Ray Tracer

Author: Oriana Henriquez

Date: December 6th, 2025

1. Program Overview

This project implements a deterministic CUDA-based ray tracer that computes one primary ray per pixel with direct Lambertian lighting, shadow testing, and a background sky color. The program includes a CPU reference ray tracer, a GPU CUDA ray tracer, a 2D CUDA grid and block configuration, shared memory caching of the scene per thread block, and CPU vs GPU timing and speedup analysis. The output image is written to out.ppm.

2. CUDA Kernel Configuration

The CUDA kernel is launched using a 2D grid and 2D thread block configuration where each thread computes one pixel. A 16×16 block size is used. This mapping ensures one thread per pixel, coalesced framebuffer writes, and good streaming multiprocessor occupancy on the GPU.

3. Shared Memory Usage

The scene geometry is cached into shared memory once per block using an extern shared Sphere array. Each block cooperatively loads the scene from global memory before ray tracing begins. This significantly reduces repeated global memory access during intersection testing and improves effective memory bandwidth.

4. CPU and GPU Timing Methodology

CPU execution time is measured using `std::chrono::high_resolution_clock` around the `render_cpu` function call. GPU kernel execution time is measured using CUDA events placed immediately before and after the CUDA kernel launch. These timings provide consistent values with the recorded Nsight Systems execution timeline.

5. Measured Performance Results

All tests were performed at a resolution of 800×800 with one primary ray per pixel (1 spp). The following values were recorded from program output and confirmed through Nsight Systems profiling.

Mode	Resolution	Samples Per Pixel	Execution Time (s)
CPU	800×800	1	≈ 2.33
GPU (CUDA Kernel)	800×800	1	≈ 0.0121

The CPU execution time reflects the full `render_cpu` loop, while the GPU time corresponds to the execution of the `pathtrace_kernel` measured using CUDA events.

6. Parallel Speedup Analysis

Speedup is computed as the ratio of the CPU execution time to the GPU execution time. Using the measured values, the observed speedup is approximately:

$$\text{Speedup} = 2.33 / 0.0121 \approx 192\times$$

This result demonstrates nearly two orders of magnitude performance improvement using GPU parallelization.

7. Nsight Systems Profiling Results

Nsight Systems was used to profile application-level execution. The timeline clearly shows a long CPU execution region corresponding to `render_cpu` and a short GPU execution region corresponding to the CUDA kernel. The CUDA kernel execution time closely matches the GPU time measured by CUDA events (~ 12 ms), confirming minimal profiling overhead and efficient kernel execution.

8. Nsight Compute Kernel Analysis

Nsight Compute was used to analyze the performance of the CUDA ray tracing kernel. Key observations include high achieved occupancy due to the 16×16 block configuration, moderate register usage, and efficient utilization of shared memory. After the cooperative scene load, most intersection tests access shared memory rather than global memory, significantly improving kernel throughput.

9. Optimization Summary

The performance gains observed in this project are attributed to the following optimizations:

- Mapping one CUDA thread per image pixel.
- Cooperative loading of the scene into shared memory.
- Coalesced memory accesses for framebuffer writes.
- Deterministic single-bounce ray tracing that minimizes branch divergence.

10. Conclusion

This PA4 CUDA ray tracing implementation demonstrates the effectiveness of GPU parallelization for graphics applications. With an observed speedup of approximately $192\times$ over the CPU implementation, the results confirm that combining a well-structured CUDA grid with shared memory optimization leads to significant performance gains. Nsight

profiling verifies that kernel execution and memory behavior are efficient and well-balanced.